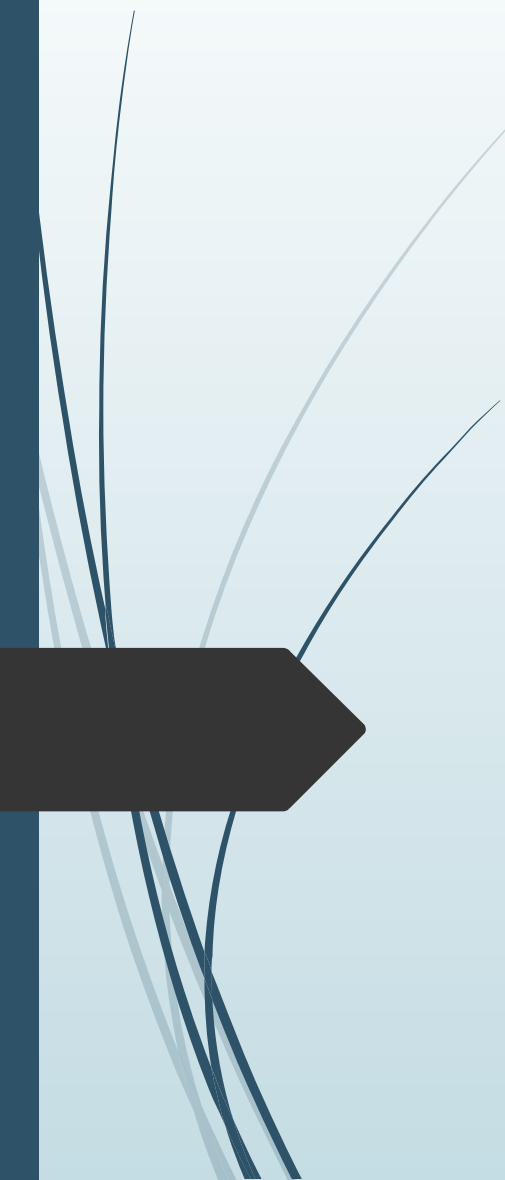


Introduction to Artificial Intelligence

Week 6



Learning by Searching

Part II

Exploring and Exploiting

- Exploration – the want for a search method to find new areas of the search space which have not been visited yet
- Exploitation – the want for a search method to locally search about the best known areas in the space in order to refine these current solutions
- All search algorithms will have some mix of both properties
- Note that if we only have a limited number of objective tests available, then we have a zero-sum game – we can explore or we can search with a test

Exploration vs Exploitation

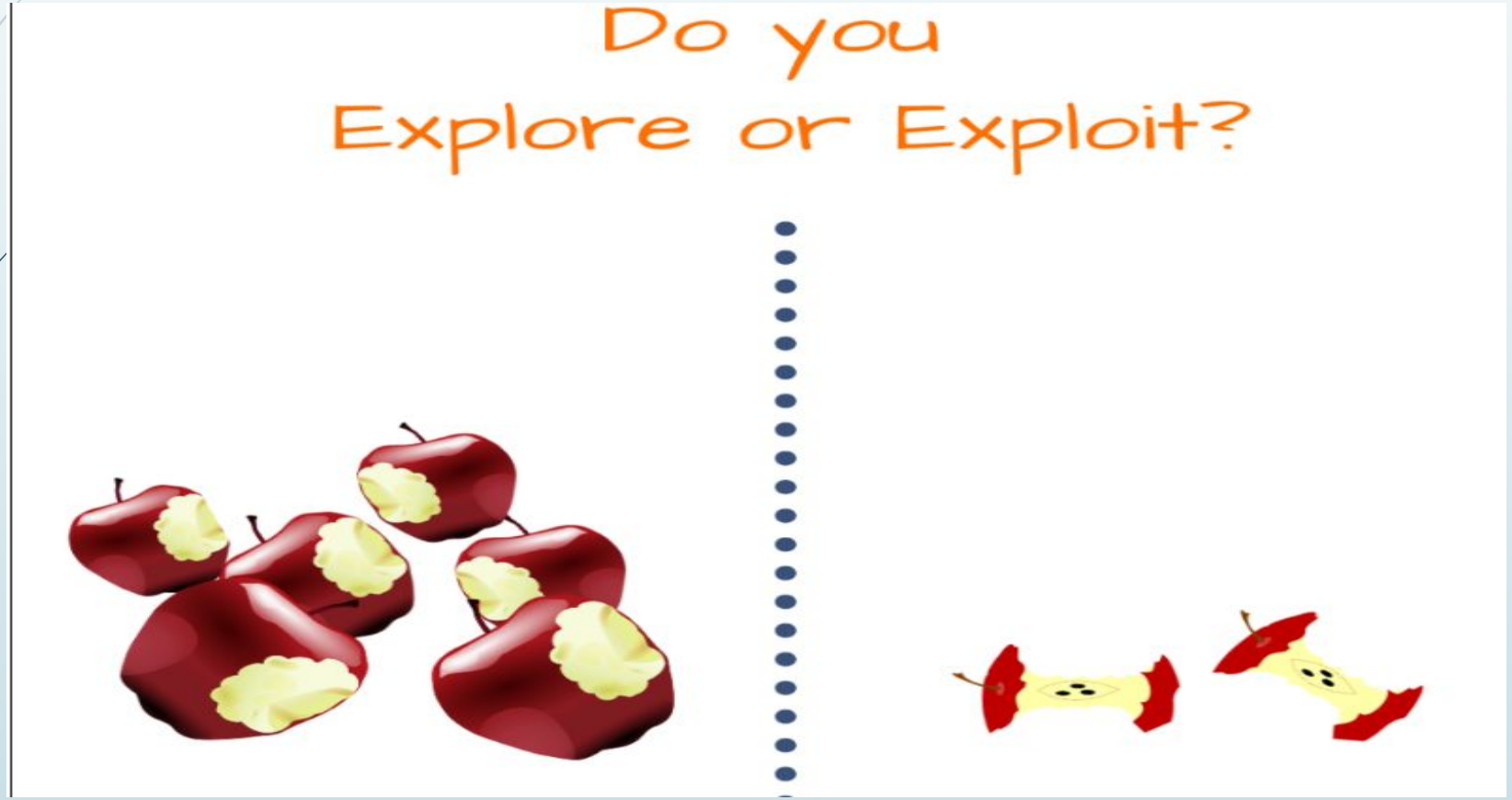
- Before visiting a new city, we searched for the restaurants where we can buy sushi.

Exploration

- Before visiting a new city, we searched for the restaurants where we can buy sushi.
- It's a new city, we have **no information** about it before.

Exploration vs Exploitation

Where is the worm?



Exploration vs Exploitation

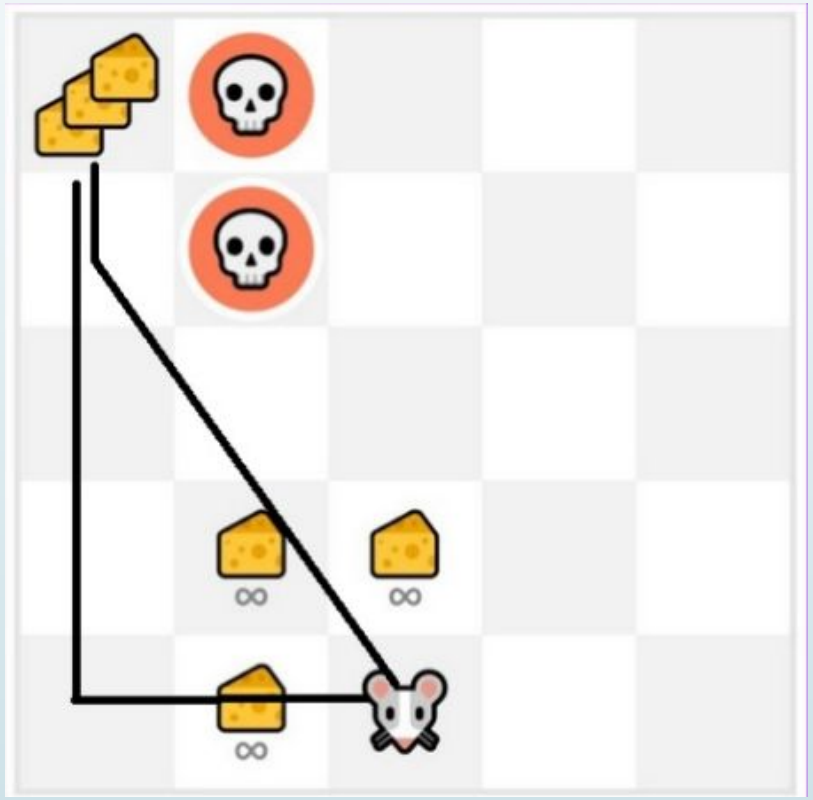
- In this game, our mouse can have an infinite amount of small cheese (+1 piece per day). But at the top of the maze, there is a gigantic sum of cheese (+10000).

- Now, let's delve into the motivation behind the possible mouse's actions: What motivates the mouse to explore the maze further, and under what conditions would it choose to do so? How does the desire for a better solution or a change in satisfaction level influence its decision to explore or exploit the available cheese sources?

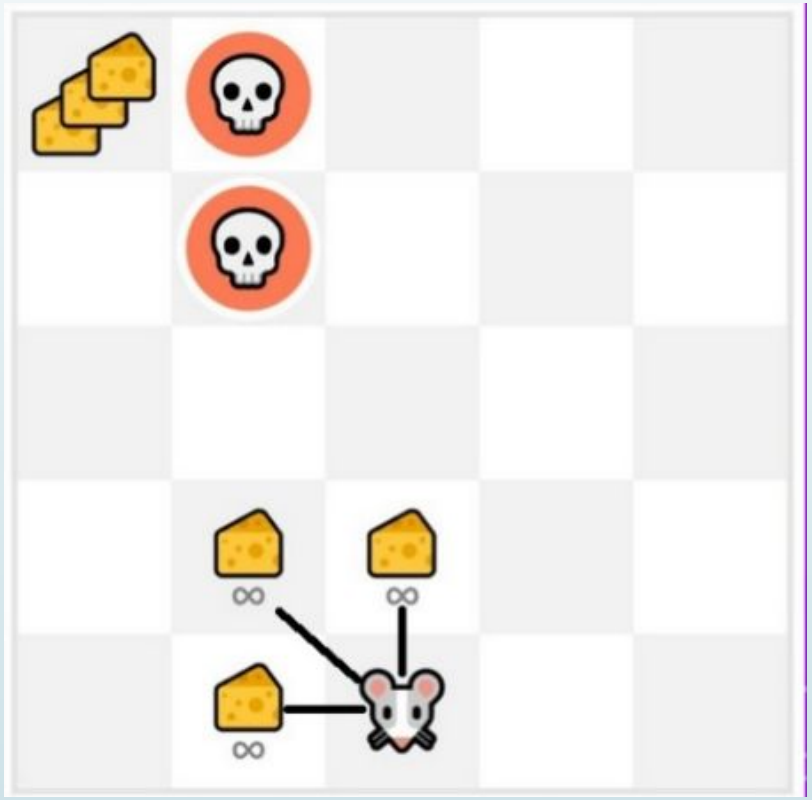
Exploration vs Exploitation

- If we only focus on exploitation, our agent will never reach the gigantic sum of cheese. Instead, it will only exploit the nearest source of rewards, even if this source is small (exploitation).
- But if our agent does a little bit of exploration, it can discover the big reward (the pile of big cheese).
- This is what we call the exploration/exploitation trade-off. We need to balance how much we explore the environment and how much we exploit what we know about the environment.

Exploration



Exploitation



Help the Prince to find Cinderella

- The prince will be at some position and the surrounding boxes will have some costs:
  - Diagonals cost = **14**
  - Non-Diagonals cost = **10**

14	10	14
10	Prince	10
14	10	14

Help the Prince to find Cinderella

- There will be three costs in each square (**Total**, G, H):
  - Cost of square from the Prince = G
  - Cost of square from Cinderella = H (straight line distance)
  - Total cost (F) = G + H
  - Our goal is to minimize the total cost

				Cinderella	
(**62**, 14, 48)	(**48**, 10, 38)	(**42**, 14, 28)			
(**62**, 10, 52)	Prince	(**48**, 10, 38)			
(**70**, 14, 56)	(**62**, 10, 52)	(**62**, 14, 48)			

Help the Prince to find Cinderella

- Prince will move to the square with least costs.
- After moving to next square, new costs will be calculated, and Prince will move to the one with least cost.

				Cinderella	
	(**62**, 28, 34)	(**48**, 24, 24)	(**42**, 28, 14)		
(**62**, 14, 48)	(**48**, 10, 38)	(**42**, 14, 28)	(**48**, 24, 24)		
(**62**, 10, 52)	Prince	(**48**, 10, 38)	(**62**, 28, 34)		
(**70**, 14, 56)	(**62**, 10, 52)	(**62**, 14, 48)			

Help the Prince to find Cinderella

- Prince will move to the square with least costs.
- After moving to next square, new costs will be calculated, and Prince will move to the one with least cost.

				Cinderella	
	(**62**, 28, 34)	(**48**, 24, 24)	(**42**, 28, 14)		
(**62**, 14, 48)	(**48**, 10, 38)	(**42**, 14, 28)	(**48**, 24, 24)		
(**62**, 10, 52)	Prince	(**48**, 10, 38)	(**62**, 28, 34)		
(**70**, 14, 56)	(**62**, 10, 52)	(**62**, 14, 48)			

Help the Prince to find Cinderella

- Prince will move to the square with least costs.
- After moving to next square, new costs will be calculated, and Prince will move to the one with least cost.

		(**62**, 42, 20)	(**48**, 38, 10)	(**42**, 42, 0)	
	(**62**, 28, 34)	(**48**, 24, 24)	(**42**, 28, 14)	(**48**, 38, 10)	
(**62**, 14, 48)	(**48**, 10, 38)	(**42**, 14, 28)	(**48**, 24, 24)	(**62**, 42, 20)	
(**62**, 10, 52)	Prince	(**48**, 10, 38)	(**62**, 28, 34)		
(**70**, 14, 56)	(**62**, 10, 52)	(**62**, 14, 48)			

Help the Prince to find Cinderella

- If there is some obstacle, the prince will change it's direction based on the scores which are explored:
  - i.e. it will move to the square with least costs.
  - If there are several with same least cost, then it will go to the one with least cost from Cinderella.

				Cinderella	
(**62**, 14, 48)	(**48**, 10, 38)	(**42**, 14, 28)			
(**62**, 10, 52)	Prince	(**48**, 10, 38)			
(**70**, 14, 56)	(**62**, 10, 52)	(**62**, 14, 48)			

Help the Prince to find Cinderella

- We will move to the square with least cost, i.e. (**42**,14,28)

				Cinderella	
(**62**, 14, 48)	(**48**, 10, 38)	(**42**, 14, 28)	(**48**, 24, 24)		
(**62**, 10, 52)	Prince	(**48**, 10, 38)	(**62**, 28, 34)		
(**70**, 14, 56)	(**62**, 10, 52)	(**62**, 14, 48)			

Help the Prince to find Cinderella

- We will move to the square with least total cost and in this case we have three of such squares, so we will move to the square with least total cost and least cost of heuristics (cost from Cinderella), i.e. (**48**,24,24), and calculate new scores

				Cinderella	
				(**48**, 38, 10)	
(**62**, 14, 48)	(**48**, 10, 38)	(**42**, 14, 28)	(**48**, 24, 24)	(**54**, 34, 20)	
(**62**, 10, 52)	Prince	(**48**, 10, 38)	(**62**, 28, 34)	(**68**, 38, 30)	
(**70**, 14, 56)	(**62**, 10, 52)	(**62**, 14, 48)			

Help the Prince to find Cinderella

- Now we can see that the lowest total cost is 48 for three squares, so we'll choose based on heuristics, which is (**48**, 38, 10).

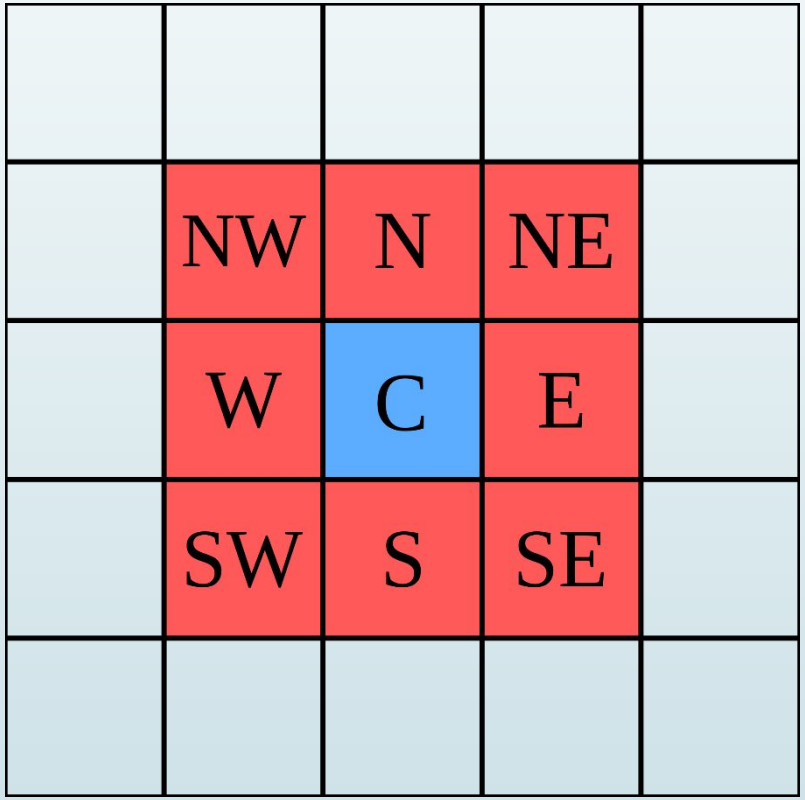
			(**62**, 52, 10)	Cinderella	(**62**, 52, 10)
				(**48**, 38, 10)	(**62**, 48, 14)
(**62**, 14, 48)	(**48**, 10, 38)	(**42**, 14, 28)	(**48**, 24, 24)	(**54**, 34, 20)	(**76**, 52, 24)
(**62**, 10, 52)	Prince	(**48**, 10, 38)	(**62**, 28, 34)	(**68**, 38, 30)	
(**70**, 14, 56)	(**62**, 10, 52)	(**62**, 14, 48)			

Help the Prince to find Cinderella

- This is our shortest path by following A* Algorithm with total cost of **48.**

			(**62**, 52, 10)	(**48**, 48, 0)	(**62**, 52, 10)
				(**48**, 38, 10)	(**62**, 48, 14)
(**62**, 14, 48)	(**48**, 10, 38)	(**42**, 14, 28)	(**48**, 24, 24)	(**54**, 34, 20)	(**76**, 52, 24)
(**62**, 10, 52)	Prince	(**48**, 10, 38)	(**62**, 28, 34)	(**68**, 38, 30)	
(**70**, 14, 56)	(**62**, 10, 52)	(**62**, 14, 48)			

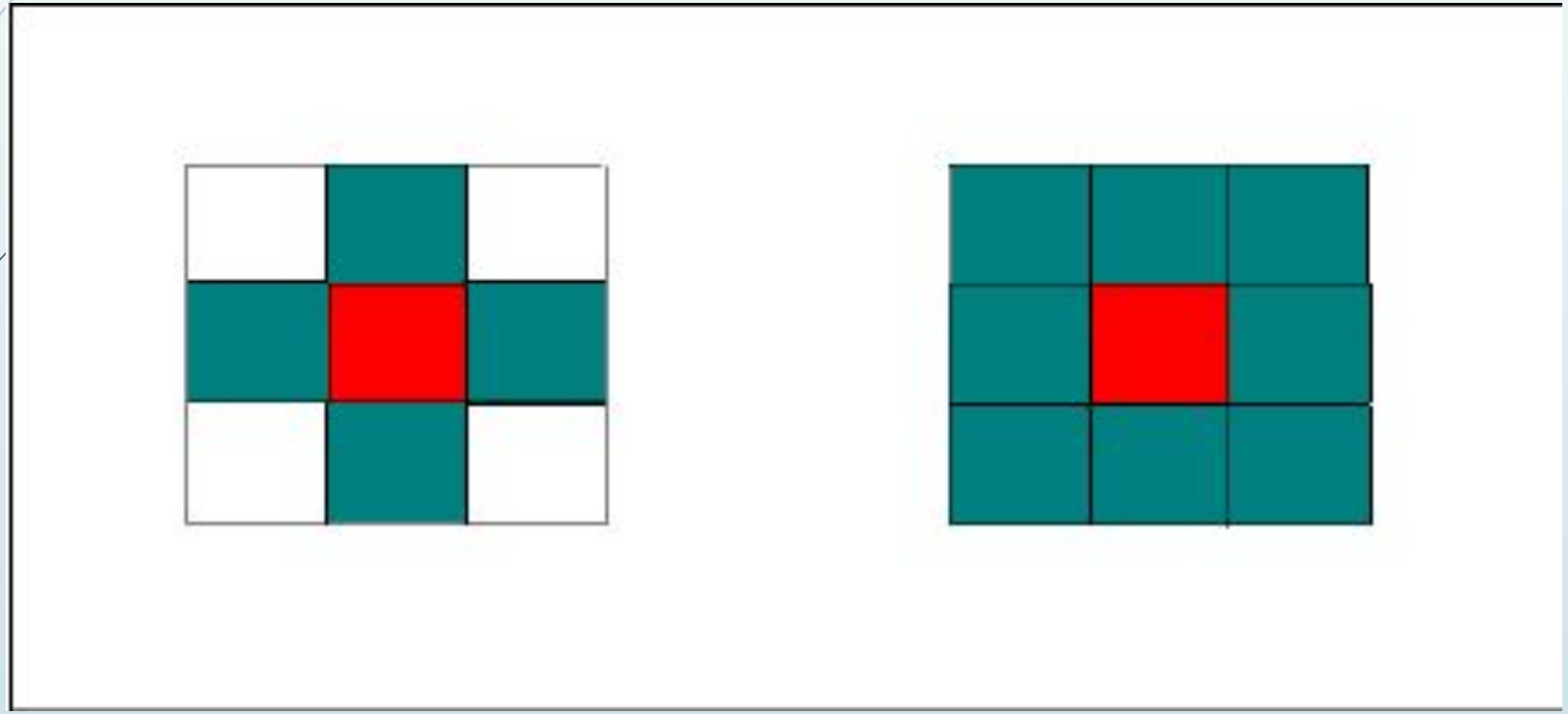
Moore Neighbourhood Algorithm Range 1



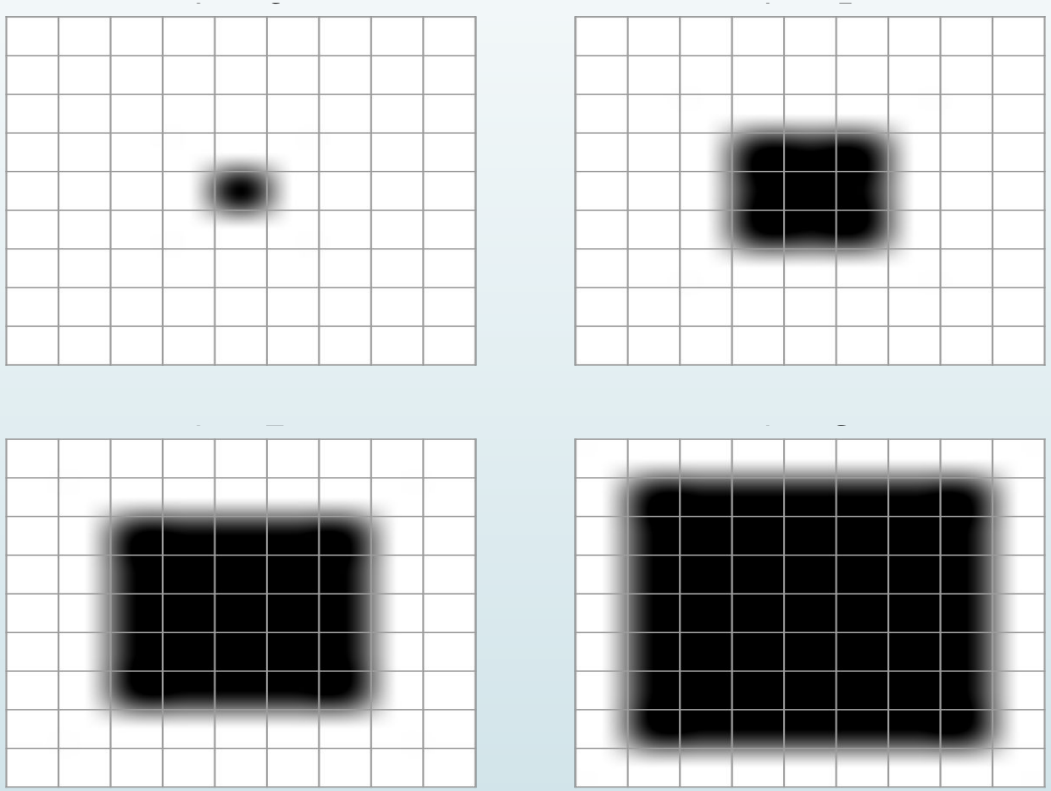
Moore Neighbourhood Algorithm

- Using Moore Neighbourhood algorithm, the agent (the prince in our example) can check all 8 cells of his immediate neighbourhood (exploring the environment around him).
- If any of these cells contain an obstacle such as the stepmom or stepsisters, the prince will be aware of them.
- Based on this perception, the prince can decide how to move in order to avoid coming into a cell containing an obstacle.

Von Neumann vs Moore Range 1

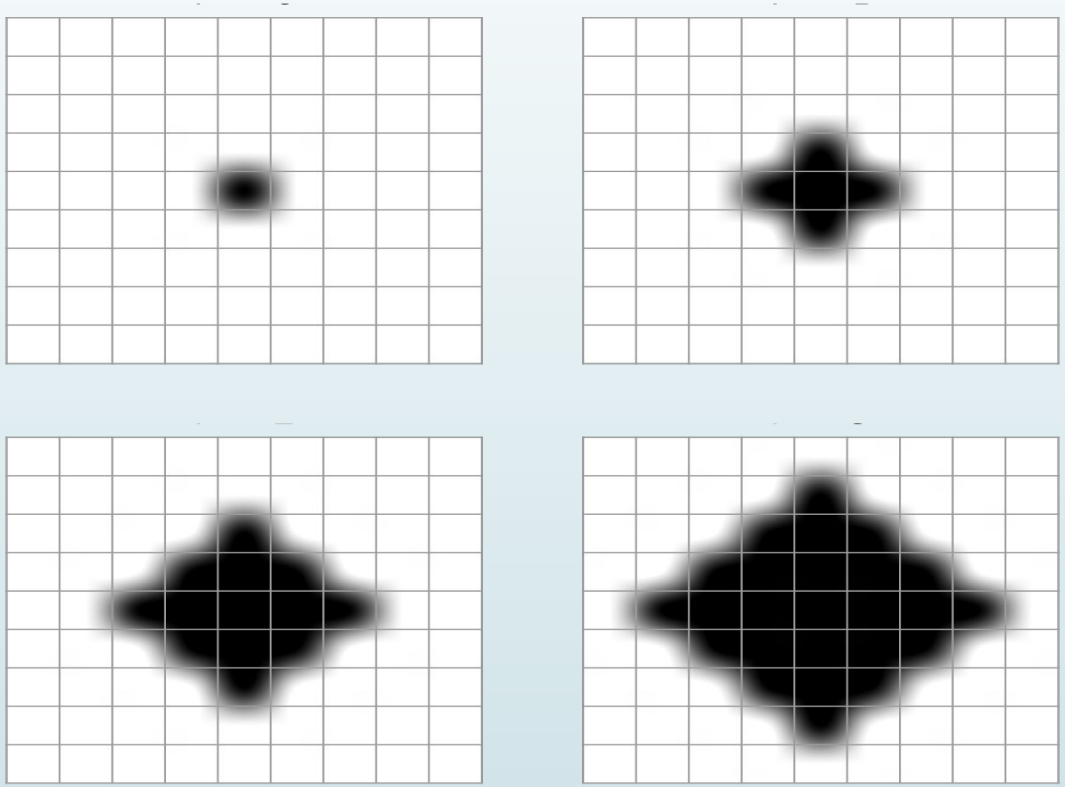


Moore Neighbourhood Ranges



Moore neighbourhoods for ranges r = 0, 1, 2, and 3 are illustrated above. The number of cells in the Moore neighbourhood of range r is (2r+1)^2, the square of odd number

Von Neumann Neighbourhood Ranges



Von Neumann neighbourhoods for ranges r = 0, 1, 2, and 3 are illustrated above. The number of cells in the von Neumann neighbourhood of range r is 2r(r+1)+1

Task 1: Help Prince

- Divide into teams of 4-5 people and find out the shortest path for Prince to meet Cinderella. Follow the A* algorithm and write down the costs at each visited square. Stepmom, Stepsisters are obstacles for the Prince.

9								
8					Cinderella			
7								
6				Stepsis1	Stepsis2	Stepsis3		
5								
4	Stepmom							
3								
2		Prince						
1								
	1	2	3	4	5	6	7	8

Task 2: Help Prince

- Repeat the same game with obstacles. Prince is able to perceive obstacles via Moore neighbourhood. The **obstacles** are hidden, ask the instructor for obstacle after making a move.

9								
8					Cinderella			
7								
6								
5								
4								
3								
2		Prince						
1								
	1	2	3	4	5	6	7	8

Homework

- Implement A* algorithm for considered world with obstacles using C/C++/C#/Java/Python.